

CS201 Midterm 3

28 December 2023

Olcay Taner YILDIZ

Abstract—Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures, (iv) you can use any function given in the data structure.

I. QUESTION (GRAPH)

Write the method in linked list implementation

`Graph constructGraphFromNumbers(int`

which constructs a graph from numbers $0, 1, 2, \dots, N - 1$; where the numbers represent the node indexes and two nodes are connected if they have common divisor other than 1.

II. QUESTION (GRAPH)

Write the method in array implementation

`int numberOfCompleteSubGraphs()`

which returns the number of complete subgraphs in the graph. A complete graph is a graph, in which all vertices are connected to all vertices. Assume that the graph only consists of complete subgraphs of size > 1 , there are no extra vertices, which is not in a complete subgraph. You are not allowed to use depth first search or breadth first search. In the graph below $(1, 2, 5)$, $(3, 6)$ and $(4, 7)$ are complete subgraphs.

	1	2	3	4	5	6	7
1	0	1	0	0	1	0	0
2	1	0	0	0	1	0	0
3	0	0	0	0	0	1	0
4	0	0	0	0	0	0	1
5	1	1	0	0	0	0	0
6	0	0	1	0	0	0	0
7	0	0	0	1	0	0	0

III. QUESTION (DISJOINT SET)

Write the method in **DisjointSet** class

`int* getSetWithIndex(int index)`

which returns the indexes of all sets in the disjoint set where a set with index *index* is in that set.

IV. QUESTION (HEAP)

Given the index of a heap node in a binary max heap, write the method in **MaxHeap** class

`int* ascendants(int index)`

that returns the indexes of its ascendants (parent, grandparent, grandgrandparent, ...) of this node. The array should contain only that many items not more not less.

V. QUESTION (HEAP)

Write the method in **MinDHeap** class

`int numberOfPlacesToReplace(int key)`

that finds the number of places in the heap, which can be replaced with the given value key.

VI. QUESTION (DISJOINT SET)

Write the method in **DisjointSet** class

`void unionOfSets(int* indexList, int N)`

that merges N sets given their indexes in the `indexList`. You should use `findSet` and the original `unionOfSets` method. Merge the sets such that the resulting merged set will have the minimum depth. Use an algorithm that sorts the sets according to their depths.